

I-PARAKEET: INTEGER-ONLY CONFORMER ASR ON MOBILE NPU

Taichi Nishimura

Sony Interactive Entertainment, Tokyo, Japan

ABSTRACT

In this paper, we propose I-Parakeet, an integer-only implementation of NVIDIA’s Parakeet-CTC (0.6B parameters) that runs on a smartphone NPU without any floating-point operator or CPU fallback. Modern Conformer ASR models are hard to deploy on edge devices because of their size, and quantized models still fall back to floating point for numerically sensitive operations. This prevents them from fully exploiting integer accelerators such as mobile NPUs. To achieve this, our contributions are threefold. First, we derive an integer formulation of the relative-positional self-attention at the core of the Conformer. We fuse its two score branches with different quantization scales and the relative shift into integer-only operations. Second, we introduce a minimax-optimized Swish approximation that minimizes the maximum error of the Swish output. Third, a layer-wise range analysis of activations yields two targeted remedies: an INT16 grid for the BatchNorm output and percentile calibration for the heavy-tailed pre-encoder activations. I-Parakeet achieves 4.97% WER on LibriSpeech test-other, running on a Qualcomm NPU at a real-time factor of 0.048, $7.5\times$ faster than a CPU baseline.

Index Terms— automatic speech recognition, quantization, neural processing units

1. INTRODUCTION

Automatic speech recognition (ASR) has achieved remarkable reductions in word error rate (WER) driven largely by Transformer-based architectures [1, 2, 3, 4, 5]. In particular, Conformer [5], which combines self-attention with convolution to capture both global and local dependencies, has become a de-facto standard. However, these gains have come through scale. For example, NVIDIA’s Parakeet-CTC [6], a representative modern Conformer, has 0.6B parameters. At this scale, the weights alone exceed 2.4 GB in single precision, and floating-point inference strains the memory, latency, and power of edge devices, making on-device deployment challenging.

Quantization [7] addresses this deployment gap. INT8 weights reduce the model size of Parakeet-CTC 0.6B from 2.4GB to 600MB. Most importantly for edge deployment, modern mobile SoCs ship with neural processing units (NPUs) built for integer arithmetic: quantized inference can be offloaded entirely from the CPU/GPU, freeing them for other

workloads while reducing power draw.

However, most quantized ASR models are not integer-only. The INT8 weights are dequantized to FP16/FP32 to stabilize numerically sensitive operations such as normalization, softmax, and nonlinear activations. Such models cannot fully exploit NPUs and cannot run at all on processors that lack floating-point units, such as ARM Cortex-M. Kim *et al.* addressed this for language models with I-BERT [8], designing integer-only kernels for LayerNorm [9], Softmax, and GELU [10], and later extended the approach to ASR [11], demonstrating integer-only inference for CNN-[12] and Conformer-based models [5]. However, the hardware evidence across integer-only studies [8, 11] is limited to GPUs, and whether an integer-only Conformer actually runs on a mobile NPU remains unexplored.

In this paper, we propose I-Parakeet, an integer-only implementation of NVIDIA’s Parakeet-CTC that runs on the NPU of a consumer smartphone. We use integer-only as a deployment property: no floating-point operator and no CPU fallback. To deploy a modern Conformer on a real NPU, we resolve three issues. First, the relative-positional self-attention at the core of the Conformer has not been explicitly formulated for integer-only inference. Its two score branches have different quantization scales, and the relative shift interleaves them with zero-padding. We derive an integer-only formulation that fuses the two branches and the relative shift (Section 3.1). Second, the Swish activations must be approximated in integer arithmetic. Prior work [8] fits the coefficients to the sigmoid or tanh rather than to the Swish output. We instead minimize the maximum error of the Swish output and show that this criterion matters for WER (Section 3.2). Third, several tensors exhibit activation ranges that INT8 cannot cover. Although naive INT8 quantization degrades Parakeet-CTC significantly, moving every tensor to a wider grid harms the NPU’s throughput. A layer-wise range analysis assigns an INT16 grid to the BatchNorm output and percentile calibration to the pre-encoder, while other tensors stay INT8 (Section 3.3).

2. I-PARAKEET OVERVIEW

2.1. Parakeet-CTC

Fig. 1 (a) shows an overview of Parakeet-CTC-0.6B [13], a Conformer-based CTC model consisting of a convolutional

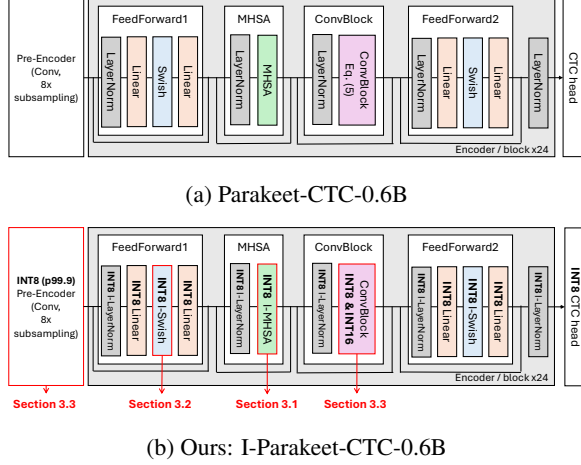


Fig. 1: Model Overview. The integer relative-position self-attention, integer Swish, and quantization strategy in the pre-encoder/ConvBlock are described in Section 3.1, 3.2, and 3.3.

pre-encoder, a stack of $N=24$ Conformer blocks (encoder), and a linear CTC head. Given the input log-mel spectrogram $M \in \mathbb{R}^{80 \times T_0}$ (T_0 is the number of input frames), the pre-encoder firstly reduces its frame rate by a factor of 8 through three strided convolutions with ReLU activations. Then it projects the result to the model dimension $d = 1024$ with a linear layer. This yields a feature sequence $\mathbf{H}^{(0)} = [\mathbf{h}_1, \dots, \mathbf{h}_L]^\top \in \mathbb{R}^{L \times d}$, where $L = \lceil T_0/8 \rceil$ and $\mathbf{h}_l \in \mathbb{R}^d$ is the feature vector of the l -th frame.

To map $\mathbf{H}^{(0)}$ to a representation suitable for CTC decoding, the encoder applies a stack of conformer blocks. Each block transforms $\mathbf{X} = \mathbf{H}^{(i-1)}$ into $\mathbf{H}^{(i)}$ through four residual sub-modules:

$$\mathbf{Z}_1 = \mathbf{X} + 0.5 \text{FFN}_1(\text{LN}(\mathbf{X})), \quad (1)$$

$$\mathbf{Z}_2 = \mathbf{Z}_1 + \text{MHSA}(\text{LN}(\mathbf{Z}_1)), \quad (2)$$

$$\mathbf{Z}_3 = \mathbf{Z}_2 + \text{Conv}(\text{LN}(\mathbf{Z}_2)), \quad (3)$$

$$\mathbf{H}^{(i)} = \text{LN}(\mathbf{Z}_3 + 0.5 \text{FFN}_2(\text{LN}(\mathbf{Z}_3))), \quad (4)$$

where the ConvBlock Conv in Eq (3) is a five-stage pipeline:

$$\text{Conv}(\mathbf{U}) = \text{PW}_2(\text{Swish}(\text{BN}(\text{DW}(\text{GLU}(\text{PW}_1(\mathbf{U}))))), \quad (5)$$

with $\text{PW}_{1,2}$ pointwise convolutions, GLU the gated linear unit, DW a depthwise convolution ($K=9$), and BN BatchNorm. In Eq (1)–Eq (4), LN denotes LayerNorm, $\text{FFN}_{1,2}$ are feed-forward networks with Swish activations [14], and MHSA is multi-head self-attention with relative positional encoding [15]. Finally, the CTC head projects each frame to a vocabulary size and greedy decoding produces the transcription.

2.2. Quantization of Parakeet-CTC

Fig. 1(b) shows I-Parakeet, which runs the entire network of Parakeet-CTC in integer arithmetic. Every tensor uses uniform

symmetric quantization. A real tensor $\mathbf{x}$ is mapped to its b -bit integer tensor $\mathbf{q}_x$ as

$$\mathbf{q}_x = \lfloor \text{clip}(\mathbf{x}, -\alpha_x, \alpha_x) / S_x \rfloor, \quad S_x = \alpha_x / (2^{b-1} - 1). \quad (6)$$

The clipping range α_x is a scalar computed offline from calibration data (LibriSpeech dev-other). The common choice is the min–max range $\alpha_x = \max |\mathbf{x}|$ over the calibration set, which covers every observed value, but a few outliers spread the range and degrade the rounding precision. A percentile of $|\mathbf{x}|$ (e.g., 99.9%) is a robust alternative that clips the tail instead. The choice of α_x significantly affects WER and is discussed in Section 3.3. While activations use one α_x per tensor, weights have α_x per channel, thus Eq (6) is applied independently channel by channel. The bit width b is 8 unless stated otherwise. A linear layer $\mathbf{y} = \mathbf{W}\mathbf{x}$ then needs only integer operations. The product $\mathbf{q}_W \mathbf{q}_x$ accumulates in INT32 and is requantized to the grid of $\mathbf{y}$:

$$\mathbf{q}_y = \lfloor \frac{S_W S_x}{S_y} \mathbf{q}_W \mathbf{q}_x \rfloor = \lfloor 2^{-n} m \mathbf{q}_W \mathbf{q}_x \rfloor, \quad (7)$$

where the fixed-point multiplier $m = \lfloor 2^n S_W S_x / S_y \rfloor$ is precomputed offline ($n=16$). Since the real-valued ratio $S_W S_x / S_y$ never appears at run time, a linear layer reduces to an integer matrix product, an integer multiplication by m , and a right n bit shift, with no floating-point operation and no division [16]. This is what we call integer-only arithmetic. Convolutions are linear as well and follow the same rule. BatchNorm is folded into the preceding depthwise convolution. For LayerNorm and Softmax we use the integer kernels proposed in I-BERT [8].

To realize I-Parakeet, this paper addresses three issues that are not treated in the previous work. First, we derive an integer relative-positional MHSA, whose scores are the sum of two products with different scales and cannot be requantized by Eq (7) directly (Section 3.1). Second, we propose an integer Swish approximation and show that minimizing the maximum error of the Swish output matters for WER (Section 3.2, 4.2). Third, a layer-wise analysis of activations finds a few tensors whose range a single INT8 grid cannot cover: the BatchNorm output and the pre-encoder activations. We spread the grid from $b=8$ to 16 for the former and squeeze α_x to the 99.9th percentile for the latter, while all other tensors stay INT8 (Section 3.3).

3. METHOD

3.1. Integer Relative-Positional Self-Attention

In this section, we derive an integer-only formulation of the relative-positional MHSA in Parakeet. We first describe its floating-point definition. For one head, let $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{L \times d_k}$ be the query, key, and value obtained by linear projections of the block input, and let $\mathbf{P} \in \mathbb{R}^{(2L-1) \times d_k}$ be the projection of the sinusoidal relative-position embeddings. With learned

biases $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{d_k}$, the attention scores decompose into a content branch $\mathbf{A}^c$ and position branch $\mathbf{A}^p$:

$$\mathbf{A}^c = (\mathbf{Q} + \mathbf{1}\mathbf{u}^\top) \mathbf{K}^\top, \mathbf{A}^p = \Phi((\mathbf{Q} + \mathbf{1}\mathbf{v}^\top) \mathbf{P}^\top), \quad (8)$$

$$\mathbf{A} = \text{softmax}((\mathbf{A}^c + \mathbf{A}^p) / \sqrt{d_k}), \mathbf{O} = \mathbf{A}\mathbf{V} \quad (9)$$

where Φ is the relative-shift operation that realigns the $(2L-1)$ relative positions to the L absolute key positions. An integer-only implementation must fix one INT8 grid per tensor at compile time. The two branches are calibrated independently, $\mathbf{A}^c \approx S_c \mathbf{q}_c$ and $\mathbf{A}^p \approx S_p \mathbf{q}_p$ with $S_c = S_Q S_K$ and $S_p = S_Q S_P$. Since $S_c \neq S_p$ in general, the sum $\mathbf{q}_c + \mathbf{q}_p$ does not represent $\mathbf{A}^c + \mathbf{A}^p$. Therefore, the two tensors must be converted to a common grid before the addition in Eq (9).

To resolve this, we fuse the conversion, addition, and the $1/\sqrt{d_k}$ scaling into a single requantization. Given the grid of fused scores S_s , the fused integer scores are computed as:

$$\mathbf{q}_s = \left\lfloor \frac{S_c}{S_s \sqrt{d_k}} \mathbf{q}_c + \frac{S_p}{S_s \sqrt{d_k}} \Phi(\mathbf{q}_p) \right\rfloor \quad (10)$$

$$= \left\lfloor 2^{-n} (m_c \mathbf{q}_c + m_p \Phi(\mathbf{q}_p)) \right\rfloor, \quad (11)$$

where $m_c = \lfloor 2^n S_c / (S_s \sqrt{d_k}) \rfloor$ and $m_p = \lfloor 2^n S_p / (S_s \sqrt{d_k}) \rfloor$. The softmax and the product with $\mathbf{V}$ follow I-BERT and Eq (7), so the whole MHSA runs in integer arithmetic. For efficient inference, we note two points on the position branch. First, $\mathbf{P}$ depends only on the sequence length, so we quantize it offline and store $\mathbf{q}_P$ as an INT8 constant. Second, Φ only moves values and inserts zeros without any arithmetic. Since moving values does not change them, Φ gives the same result before and after quantization: $\Phi(S \mathbf{q}) = S \Phi(\mathbf{q})$. Hence, we apply Φ to the integer tensor $\mathbf{q}_p$ directly, which we implement as a static index map.

3.2. Minimax-Optimized Integer Swish

Parakeet-CTC has the Swish activation $\text{sw}(x) = x \sigma(x)$ in every feed-forward and ConvBlock layer, and the sigmoid σ has no integer kernel. As with I-BERT [8], we approximate σ with a second-order polynomial. Since $\sigma(x) = (1 + \tanh(x/2))/2$, we approximate $\tanh$, which is an odd function, thus is enough to fit the polynomial on $u \geq 0$ and mirror it to $u < 0$. Following previous work, we use the polynomial $a(u - c)^2 + 1$, whose vertex at $u = c$ takes the value 1 (the saturated value of $\tanh$) and clip the output to 1 for $u > c$:

$$\widehat{\tanh}(u) = \text{sgn}(u) \left[a (\min(|u|, c) - c)^2 + 1 \right]. \quad (12)$$

I-BERT adopts least-squares fit to optimize (a, c) to minimize the mean squared error (L_2) between $\widehat{\tanh}$ and $\tanh$.

However, we found that this approach is sub-optimal for Parakeet-CTC due to two issues. First, the L_2 treats all x equally, whether x is near 0 or far from it, but in Swish, the

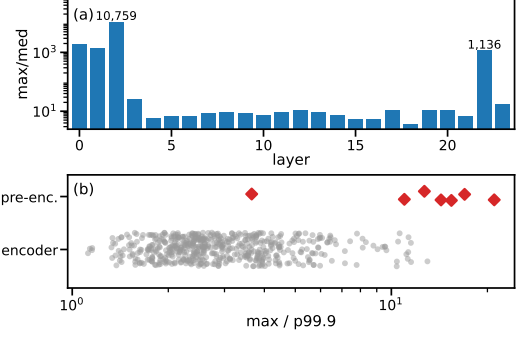


Fig. 2: (a) Per-channel spread of the BatchNorm scale $\gamma_c / \sqrt{\sigma_c^2 + \epsilon}$ per layer (max/median). (b) Ratio of the observed max to the 99.9 percentile for all calibrated activations.

σ is multiplied by x , causing the big error on large $|x|$ (e.g., The same σ error is 40 times larger at $x = 4$ than at $x = 0.1$). Based on this, we assume that minimizing the maximum error (L_∞) over x is better than the average L_2 . Second, although I-BERT fits the $\tanh$, the coefficients that best fit the $\tanh$ are not necessarily the ones that best fit the Swish output. To this end, we focus on fitting the Swish output directly to reduce the error at the output.

Based on them, we optimize (a, c) that minimizes the maximum error of the Swish output:

$$(a^*, c^*) = \arg \min_{a, c} \max_x |\text{sw}(x) - \widehat{\text{sw}}(x; a, c)|, \quad (13)$$

where $\widehat{\text{sw}}(x; a, c) = x (1 + \widehat{\tanh}(x/2))/2$. We solve Eq (13) numerically and obtain $a^* = -0.1240$ and $c^* = 2.4632$.

3.3. Layer-wise Activation Range Analysis

We profile the ranges of all activations over the calibration data and find that the default settings (bit-width $b = 8$ and min-max range α_x for every tensor) fails at two places due to outliers [17, 18]. Fig. 2 summarizes the analysis. We change b at one place and α_x at the other, and keep the default elsewhere.

The first failure is the BatchNorm output in the ConvBlock. Fig. 2 (a) shows the ratio of the max to the median of the BatchNorm coefficient $\gamma_c / \sqrt{\sigma_c^2 + \epsilon}$ across channels. The ratio exceeds four orders of magnitude in several layers. A per-tensor INT8 grid must cover the few extreme channels, so the remaining channels are left with almost no quantization levels. We therefore set $b = 16$ for the BatchNorm output. In Table 3 we call this setting per-tensor INT16, and the default per-tensor INT8; both names refer to the BatchNorm only.

The second failure is the pre-encoder activations. Fig. 2 (b) shows the ratio of the max to the 99.9th percentile of the activation values. In the pre-encoder the ratio reaches $11\text{--}21\times$, so a min-max α_x spends most of the INT8 grid on a 0.1% tail. Inside the encoder it stays at $2\text{--}3\times$. Therefore we set α_x to the 99.9th percentile in the pre-encoder and keep min-max in the encoder. In Table 3 we call this hybrid scaling.

Table 1: On-device comparison on LibriSpeech test-other (Nothing Phone (3a)). The unit of peak memory is MiB.

Method	Backend	Weights / compute	WER	RTF	Peak memory
parakeet.cpp [19]	CPU	FP16 / FP32	3.76	0.36	1517
parakeet.cpp [19]	CPU	INT8 / FP32	3.76	0.42	1045
Stock FP16	NPU	FP16 / FP16	100.0	–	–
Stock INT8 PTQ	NPU	INT8 / INT8	100.0	–	–
I-Parakeet	NPU	INT8 / INT8 & INT16	4.97	0.048	612

4. EXPERIMENTS

4.1. On-Device Results

Settings. We evaluate WER, RTF, and peak memory on a mid-range consumer smartphone (Nothing Phone (3a), Snapdragon 7s Gen 3, SM7635) with QNN SDK (QAIRT 2.47) as the NPU runtime. Since the NPU requires static shapes, we follow prior work [20] and compile graphs for input lengths of 3 to 35 seconds, routing each utterance to the smallest graph that contains it. Padding with silence features adds 23% to the processed frames, and all RTFs include this overhead. WER is computed with the Whisper English text normalizer. The baselines are parakeet.cpp [19] on two CPU threads with FP16 and q8.0 weights (both with FP32 compute), and the same checkpoint converted by the stock Qualcomm SDK toolchain to FP16 and to INT8 by post-training quantization.

Results. Table 1 summarizes the results. I-Parakeet runs entirely on the NPU without CPU fallback, $7.5\times$ faster than FP16 parakeet.cpp with 60% less peak memory, at a WER cost of 1.21 points. Weight-only INT8 does not speed up the CPU path, so the gain comes from integer compute on the NPU, not from smaller weights. The on-device WER of 4.97% is slightly below the 5.32% of our integer simulator (Section 4.2) because the NPU evaluates the sigmoid in Swish with a hardware lookup table instead of the polynomial. Both stock paths give 100% WER: the FP16 failure is consistent with the pre-encoder activations of Fig. 2 exceeding the FP16 range, and the INT8 failure shows that tool-supported post-training quantization is not sufficient for this model.

4.2. Accuracy Analysis in Simulation

Settings. To isolate each design choice, we use a PyTorch implementation of the integer arithmetic that mirrors the deployed graph but has Swish polynomial approximation in Section 3.2. We evaluate on LibriSpeech [21] and Common Voice [22]. We compare I-Parakeet with two integer-only baselines that share the integer attention of Section 3.1. I-BERT recipe [8, 11] quantizes every tensor to INT8 with min–max ranges and approximates Swish by the least-squares fit to the tanh. Naive INT8 uses the same INT8 min–max quantization but our minimax Swish, thus the difference between Naive INT8 and I-Parakeet is the two range remedies of Section 3.3.

Results. Table 2 reports WER on both datasets. I-Parakeet

Table 2: Simulated WER on PyTorch; on-device results are in Table 1.

Model	LibriSpeech		Common Voice
	test-clean	test-other	test
Parakeet-CTC (FP32)	1.87	3.76	10.55
I-BERT recipe [8, 11]	3.41	7.41	19.05
Naive INT8	3.01	6.38	16.54
I-Parakeet	2.61	5.32	14.70

Table 3: Ablation on LibriSpeech test-other (WER). Max err. is the max absolute error of the approximated Swish output.

Configuration	Max err.	WER
<i>Swish approximation (Sec. 3.2)</i>		
L_∞ fit to Swish (ours)	0.039	5.32
L_2 fit to Swish	0.045	5.49
L_∞ fit to tanh	0.068	5.81
L_2 fit to tanh	0.073	5.95
Hard-Swish	0.142	6.33
<i>BatchNorm output precision (Sec. 3.3)</i>		
per-tensor INT16 (ours)	–	5.32
per-channel INT16 (not deployable)	–	5.29
per-tensor INT8 (min–max)	–	5.91
<i>Scale calibration on pre-encoder and encoder (Sec. 3.3)</i>		
hybrid scaling of min–max and p99.9 (ours)	–	5.32
min–max only	–	5.64
p99.9 only	–	8.03

reaches 2.61% on test-clean and 5.32% on test-other, 0.74/1.56 points behind FP32. Compared with Naive INT8, it recovers 0.40/1.06 points, the contribution of the two range remedies, and the same ordering holds on Common Voice.

Ablation 1: Swish approximation. Table 3 varies one design choice at a time. The ranking of the maximum output error matches the ranking of WER. On the norm perspective, L_∞ beats L_2 for both fitting targets. In terms of fitting target function, fitting the Swish output beats fitting the tanh under both norms. Therefore, the two axes contribute independently. Hard-Swish [23] degrades WER by one point.

Ablation 2: BatchNorm output precision. Per-tensor INT8 for the BatchNorm output degrades WER to 5.91. Per-channel INT16 reaches 5.29 but is not supported for activations on the NPU. Our deployable per-tensor INT16 attains 5.32, within 0.03 points of per-channel INT16.

Ablation 3: Scale calibration. Hybrid scaling beats min–max everywhere. Applying p99.9 everywhere is far worse (8.03), so clipping helps only where the distribution is heavy-tailed.

5. CONCLUSION

We proposed I-Parakeet, an integer-only Parakeet-CTC-0.6B running on a smartphone NPU at an RTF of 0.048 with 4.97% WER on LibriSpeech test-other, showing that modern Conformer ASR needs no floating-point inference.

6. REFERENCES

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin, “Attention is all you need,” in *Proc. NeurIPS*, 2017.
- [2] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine Mcleavey, and Ilya Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. ICML*, 2023.
- [3] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” in *Proc. NeurIPS*, 2020.
- [4] Yifan Peng, Jinchuan Tian, William Chen, Siddhant Arora, Brian Yan, Yui Sudo, Muhammad Shakeel, Kwanghee Choi, Jiatong Shi, Xuankai Chang, Jee weon Jung, and Shinji Watanabe, “Owsm v3.1: Better and faster open whisper-style speech models based on e-branchformer,” in *Proc. Interspeech*, 2024.
- [5] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang, “Conformer: Convolution-augmented transformer for speech recognition,” in *Proc. Interspeech*, 2020.
- [6] Dima Rekesh, Nithin Rao Koluguri, Samuel Krman, Somshubra Majumdar, Vahid Noroozi, He Huang, Oleksii Hrinchuk, Krishna Puvvada, Ankur Kumar, Jagadeesh Balam, and Boris Ginsburg, “Fast conformer with linearly scalable attention for efficient speech recognition,” in *Proc. ASRU*, 2023.
- [7] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer, “A survey of quantization methods for efficient neural network inference,” in *arXiv*, 2021.
- [8] Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer, “I-bert: Integer-only bert quantization,” in *Proc. ICML*, 2021.
- [9] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton, “Layer normalization,” in *arXiv*, 2016.
- [10] Dan Hendrycks and Kevin Gimpel, “Gaussian error linear units (gelu),” in *arXiv*, 2016.
- [11] Sehoon Kim, Amir Gholami, Zhewei Yao, Nicholas Lee, Patrick Wang, Aniruddha Nrusimha, Bohan Zhai, Tianren Gao, Michael W. Mahoney, and Kurt Keutzer, “Integer-only zero-shot quantization for efficient speech recognition,” in *Proc. ICASSP*, 2022.
- [12] Samuel Krman, Stanislav Beliaev, Boris Ginsburg, Jocelyn Huang, Oleksii Kuchaiev, Vitaly Lavrukhin, Ryan Leary, Jason Li, and Yang Zhang, “Quartznet: Deep automatic speech recognition with 1d time-channel separable convolutions,” in *Proc. ICASSP*, 2020.
- [13] NVIDIA, “nvidia/parakeet-ctc-0.6b,” <https://huggingface.co/nvidia/parakeet-ctc-0.6b>.
- [14] Prajit Ramachandran, Barret Zoph, and Quoc V. Le, “Searching for activation functions,” in *Proc. ICLR*, 2018.
- [15] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le, and Ruslan Salakhutdinov, “Transformer-xl: Attentive language models beyond a fixed-length context,” in *Proc. ACL*, 2019.
- [16] Randy Yates, “Fixed-point arithmetic: An introduction,” <https://www.cpplab.net/web/dsp/fp.pdf>.
- [17] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer, “Llm.int8(): 8-bit matrix multiplication for transformers at scale,” in *Proc. NeurIPS*, 2022.
- [18] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” in *Proc. ICML*, 2023.
- [19] mudler, “<https://github.com/mudler/parakeet.cpp>,” <https://github.com/mudler/parakeet.cpp>.
- [20] Sihyeon Lee, Hojeong Lee, Sungwon Woo, Chengpo Yan, Suman Banerjee, and Seyeon Kim, “Npusper: Eliminating redundant computation for real-time whisper on mobile npus,” in *arXiv*, 2026.
- [21] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur, “Librispeech: An asr corpus based on public domain audio books,” in *Proc. ICASSP*, 2015.
- [22] Rosana Ardila, Megan Branson, Kelly Davis, Michael Kohler, Josh Meyer, Michael Henretty, Reuben Morais, Lindsay Saunders, Francis Tyers, and Gregor Weber, “Common voice: A massively-multilingual speech corpus,” in *Proc. LREC*, 2020.
- [23] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam, “Searching for mobilenetv3,” in *Proc. ICCV*, 2019.